

GATE CSE & PSU Exam Practice

Functions · Arrays · Recursion — 50 Questions with Answers

Q1.

GATE CSE 2024 (Set 2)

Consider the following C program. Assume parameters to a function are evaluated from right to left. What is the output?

```
#include <stdio.h>
int g(int p) { printf("%d", p);
return p; } int h(int q)
{ printf("%d", q); return q; }
void f(int x, int y) { g(x);
h(y); } int main() { f(g(10),
h(20)); return 0; }
```

Explanation: Arguments are evaluated right to left, so $h(20)$ runs first (prints "20", returns 20), then $g(10)$ runs (prints "10", returns 10). These results become $x=10$, $y=20$ inside f . Inside f , $g(x) = g(10)$ prints "10"; then $h(y) = h(20)$ prints "20". Reading the prints in the order they happen: 20, 10, 10, 20 -> 20101020.

Q2.

GATE CSE 2025 (Set 2)

What is the output of the following C program?

```
#include <stdio.h>
int g(int n) { return (n +
10); } int f(int n)
{ return g(n * 2); } int
main() { int sum = 0, n;
for (n = 1; n < 3; n++) sum
+= g(f(n)); printf("%d",
sum); return 0; }
```

Answer: 46

Explanation: For $n=1$: $f(1)=g(2)=12$, so $g(f(1))=g(12)=22$. For $n=2$: $f(2)=g(4)=14$, so $g(f(2))=g(14)=24$. $sum = 22 + 24 = 46$.

Q3.

GATE CSE 2025 (Set 1)

What is the output of the following C program?

```
#include <stdio.h>
void foo(int *p, int x) { *p = x; }
int main() { int *z;
int a = 20, b = 25;
z = &a; foo(z, b);
printf("%d", a);
return 0;
}
```

Answer: 25

Explanation: z stores the address of a. foo receives that address as p and does *p = x, i.e. it writes the value of x (25) directly into a's memory. So a becomes 25.

Q4.

GATE CSE 2015 (Set 1)

What is the output of the following C program?

```
#include <stdio.h> void
f1(int a, int b) { int
c;
c = a; a = b; b =
c; }
void f2(int *a, int *b)
{ int c;
c = *a; *a = *b; *b =
c; }
int main() {
int a = 4, b = 5, c = 6;
f1(a, b); f2(&b, &c);
printf("%d", c - a - b);
return 0;
}
```

Answer: -5

Explanation: f1 swaps copies of a and b (call-by-value), so it has no effect on main's a and b; they stay a=4, b=5. f2 swaps through pointers (call-by-reference), so it really swaps b and c: b becomes 6, c becomes 5. Result: c - a - b = 5 - 4 - 6 = -5.

Q5.

GATE CSE 2002

What is the value of j at the end of execution of the following C program?

```

int incr(int i)
{ static int count =
0; count = count + i;
return count; }
int main()
{ int i, j;
for (i = 0; i <= 4; i++)
j = incr(i); return 0;
}

```

Answer: 10

Explanation: count is static, so it persists across calls and keeps accumulating: after $i=0,1,2,3,4$ it becomes 0, 1, 3, 6, 10. The last value assigned to j is 10.

Q6.

GATE CSE 2024 (Set 2)

Consider the following C function. What does $fX(a)$ compute, for a null-terminated string a ?

```

int fX(char *a)
{ char *b = a;
while (*b) b++;
return b - a; }

```

Answer: It returns the length of the string a (the number of characters before the terminating $'\0'$) -- i.e. a hand-written equivalent of `strlen()`.

Explanation: b starts at the same address as a and is advanced one character at a time until it reaches the null terminator. The pointer difference $b - a$ then equals exactly the number of characters that were stepped over, which is the string's length.

Q7.

GATE CS 2014 (Set 1)

Consider the following C function, where E is an array of size elements. What does the value returned by MyX represent, in terms of the array E ?

```

int MyX(int *E, unsigned int size)
{ int Y = 0, Z, i, j, k; for (i = 0;
i < size; i++) Y = Y + E[i];
for (i = 0; i < size; i++)
for (j = i; j < size; j++)
{ Z = 0;
for (k = i; k <= j; k++)
Z = Z + E[k]; if (Z > Y)
Y = Z; } return Y;
}

```

Answer: The maximum sum among all contiguous sub-arrays of E (the classic "maximum subarray sum"), which is always at least as large as the sum of the whole array.

Explanation: Y starts out equal to the sum of the entire array. The nested loops then examine the sum Z of every contiguous sub-array E[i..j], and whenever a sub-array beats the current best, Y is updated. So Y ends up holding the best (maximum) contiguous sub-array sum.

Practice Question (original, not from an official paper)

Q8.

GATE CS 2013

An array of 200 elements is to be sorted using merge sort. The number of comparisons required in the worst case is closest to:

- (A) $200 \log 200$
- (B) 200^2
- (C) $200 \log (200^2)$
- (D) 200

Answer: 200 log 200

Explanation: Merge sort performs $\Theta(n \log n)$ comparisons in the worst case. For $n = 200$, that is closest to $200 \log 200$.

Q9.

GATE CS 2016 (Set 2)

What is the output of the following ANSI C program?

```
#include <stdio.h>
int main() { int
arr[4][4]; int i,
j;
for (i = 0; i < 4; i++) for (j =
0; j < 4; j++) arr[i][j] = 10 *
i + j; printf("%d", *((arr + 2)
+ 3)); return 0;
}
```

Answer: 23

Explanation: $*(arr+2)$ is row 2 of the array; adding 3 and dereferencing reaches $arr[2][3]$. By the fill rule $arr[i][j] = 10*i + j$, $arr[2][3] = 10*2 + 3 = 23$.

Q10.

GATE CS 2005

A program reads in 500 integers in the range [0, 100], representing the scores of 500 students. It then prints the frequency of each score strictly above 50. What is the best way for the program to store the frequencies?

- (A) An array of 50 numbers

- (B) An array of 100 numbers
- (C) An array of 500 numbers
- (D) A dynamically allocated array of 250 numbers

Answer: An array of 50 numbers

Explanation: Scores "above 50" means the values 51 through 100 -- exactly 50 distinct possible scores -- so a 50-element frequency array is enough; allocating for 100 or 500 would waste space.

Q11.

GATE CS 2004

A single array $A[1..MAXSIZE]$ is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables $top1$ and $top2$ ($top1 < top2$) mark the topmost element of each stack. If the space is to be used efficiently, the condition for "stack full" is:

- (A) $top1 + 1 = top2$
- (B) $top1 = top2 + 1$
- (C) $top1 = top2 + MAXSIZE$
- (D) $top1 = top2 - MAXSIZE$

Answer: $top1 + 1 = top2$

Explanation: The two stacks grow toward each other from opposite ends of the same array. They run out of room exactly when the gap between the two top pointers closes completely, i.e. when $top1 + 1 = top2$.

Q12.

GATE CS 1995

A list of n numbers is given. Which of the following methods can be used to find the maximum and the minimum element together in $O(n)$ time?

- (A) Tournament method
- (B) Divide and conquer
- (C) Dynamic programming
- (D) Greedy algorithm

Answer: Tournament method

Explanation: The tournament (pairwise-comparison) method pairs elements up, compares within pairs, then separately tracks the largest of the pair-maximums and the smallest of the pair-minimums, finding both max and min together in about $3n/2$ comparisons -- linear time.

Q13.

GATE CSE 2003

Given the C declarations $\text{int } *A[10], B[10][10]$; consider: I. $A[2]$ II. $A[2][3]$ III. $B[1]$ IV. $B[2][3]$. Which of these will NOT give a compile-time error if used as the left-hand side of an assignment statement?

Answer: I, II, and IV (not III)

Explanation: $A[2]$ is a single pointer value -- assignable. $A[2][3]$ dereferences that pointer to a single int -- assignable. $B[2][3]$ is a single int element of the 2-D array -- assignable. $B[1]$, however, denotes an entire row (an array), and an array is not a modifiable lvalue -- you cannot reassign it as a whole -- so III alone causes a compile-time error.

Q14.**GATE CSE 2025 (Set 1)**

Consider the following pseudocode, and let $A[0..29]$ be an array storing 30 distinct integers already arranged in descending order. How many swap operations are performed when $\text{fun}()$ is called on A ?

```
fun(int A[0...n-1])
{ for i = 0 to n-2
  for j = 0 to n-i-2

  if (A[j] > A[j+1])
    swap A[j] and
    A[j+1] }
```

Answer: 435

Explanation: This is one pass structure of bubble sort, which makes $n(n-1)/2$ comparisons in total -- for $n=30$ that is $30 \times 29/2 = 435$. Because the array starts in strictly descending order (the worst case for this ascending-sort routine), every single comparison turns out to find elements out of order, so every comparison triggers a swap. Swap count = comparison count = 435.

Q15.**Practice Question (original, not from an official paper)**

What is the output of the following C program?

```
#include <stdio.h>
int main() {
  int arr[6] = {2, 4, 6, 8, 10, 12};
  int *p = arr; p = p + 2;
  printf("%d %d", *p, *(p + 1));
  return 0;
}
```

Answer: 6 8

Explanation: p starts pointing at $\text{arr}[0]$. After $p = p + 2$, p points at $\text{arr}[2]$, which holds 6. $*(p+1)$ is then $\text{arr}[3]$, which holds 8.

Q16.**GATE CS 2011**

What is the value of $f(5)$ for the following recursive C function?

```
int f(int n) { static
int r = 0; if (n <=
0) return 1; if (n >
3) { r = n;
return f(n - 2) +
2; }
return f(n - 1) +
r; }
```

- (A) 5
- (B) 7
- (C) 9
- (D) 18

Answer: 18

Explanation: $f(5)$: $n > 3$, so r becomes 5 and it returns $f(3)+2$. $f(3)$: returns $f(2)+r = f(2)+5$. $f(2)$: returns $f(1)+5$. $f(1)$: returns $f(0)+5$. $f(0)$: $n \leq 0$, returns 1. Substituting back: $f(1)=1+5=6$, $f(2)=6+5=11$, $f(3)=11+5=16$, $f(5)=16+2=18$.

Q17

GATE 2011 (2 marks)

What is the return value of the following recursive C function when called as `foo(345, 10)`?

```
unsigned int foo(unsigned int n, unsigned int r)
{ if (n > 0)
  return (n % r + foo(n / r, r));
  else return 0;
}
```

Answer: 12

Explanation: With $r=10$, $n\%r$ peels off the last decimal digit of n on each call, and n/r drops it. So `foo(345,10)` sums the decimal digits of 345: $3+4+5 = 12$.

Q18.

GATE 2015 (2 marks)

Consider the following C function. What is printed when it is called as `foo("ABCD EFGH")`?

```
void foo(char *a)
{ if (*a && *a != ' ') { foo(a + 1); }
  putchar(*a); }
}
```

Answer: DCBA

Explanation: The function keeps recursing forward through the string until it hits the space (or the end of the string), then prints characters on the way back up the call stack -- i.e. in reverse order. It stops as soon as it reaches the space, so only "ABCD" is ever visited, and it comes out reversed as "DCBA"; "EFGH" is never touched.

Q19

GATE CS 2000

What does the following recursive C function compute, in terms of a , b and c ?

```
int Trial(int a, int b, int c)
{ if ((a >= b) && (c < b)) return b; else if (a >= b) return Trial(a, c, b);
  else return Trial(b, a, c); }
```

- (A) The maximum of a , b , c
- (B) The minimum of a , b , c
- (C) The middle value of a , b , c
- (D) None of the above

Answer: The middle value of a, b, c

Explanation: Each recursive call reorders the three arguments, narrowing things down until the condition $(a \geq b)$ && $(c < b)$ holds -- at which point b is exactly the value that is neither the largest nor the smallest of the three. The function returns the median of a , b , and c .

Q20

GATE CS 2010 (Set 1)

Consider the function below. How many total calls are made to f (including the initial call) while evaluating $f(5)$?

```
int f(int n) { if (n <= 1)
return 1; return f(n - 1) +
f(n - 2); }
```

Answer: 15

Explanation: Let $T(n)$ be the number of calls needed for $f(n)$. $T(0)=T(1)=1$ (just the base-case call itself), and $T(n) = 1 + T(n-1) + T(n-2)$ for $n > 1$. That gives $T(2)=3$, $T(3)=5$, $T(4)=9$, $T(5)=1+9+5=15$.

Q21

GATE CSE 2001

Consider the function below, called as `abc("123")` from `main()`. (a) What is printed? (b) If s is a null-terminated string of n characters, how many characters in total does `abc(s)` print?

```
void abc(char *s) { if
(s[0] == '\0') return;
abc(s + 1); abc(s + 1);
printf("%c", s[0]); }
```

Answer: (a) 3323321 (b) $2^n - 1$ characters

Explanation: Each call on a string of length n makes two recursive calls on a string of length $n-1$ before printing one character, giving the recurrence $C(n) = 2 \cdot C(n-1) + 1$ with $C(0)=0$, whose solution is $C(n) = 2^n - 1$. Tracing "123" by hand: `abc` of a 1-character string "x" just prints x; `abc("23")` prints "3","3","2"; `abc("123")` then prints `abc("23")` output, then `abc("23")` output again, then '1' -- giving "332" + "332" + "1" = "3323321", which is $7 = 2^3 - 1$ characters.

Q22

GATE CSE 2024 (Set 1)

What does the program print when the input from the command line is 1234 followed by a newline character?

```

#include <stdio.h>
void fX(); int
main() { fX();
return 0; } void
fX() { char a;
if ((a = getchar()) != '\n')
fX(); if (a != '\n')
putchar(a);
}

```

Answer: 4321

Explanation: Each recursive call has its own local variable *a* holding the character it just read, and the *putchar* for that call happens only after its recursive call has fully returned. So the characters get printed in the reverse of the order they were read -- this is a classic "reverse input via recursion" trick. Reading 1,2,3,4 and then printing on the way back up the stack gives 4,3,2,1.

Q23

GATE CSE 2026 (Set 1)

Consider the recursive functions below. What is the smallest positive integer *n* for which *foo(n)* returns 5?

```

int bar(int n) { if (n ==
1) return 0; else return 1
+ bar(n / 2); }
int foo(int n) { if (n ==
1) return 1; else return 1
+ foo(bar(n)); }

```

Answer: 65536 (i.e. 2^{16})

Explanation: *bar(n)* counts how many times *n* must be halved (integer division) to reach 1, so the smallest *n* with *bar(n)=k* is exactly 2^k . *foo* climbs one level only when its argument first reaches the smallest *n* that produced the previous *foo*-value: *foo*=1 first at *n*=1, *foo*=2 first at *n*=2, *foo*=3 first at *n*=4, *foo*=4 first at *n*=16, and so *foo*=5 first occurs at $n = 2^{16} = 65536$.

Question 24

Consider the following C function.

```

int fun ( int n ) {
int x = 1, k ;
if ( n == 1) return x ;
for( k = 1 ; k < n ; ++ k )
    x = x + fun( k ) * fun( n - k ) ;
return x ;
}

```

}

The return value of fun (5) is _____.

[GATE CSE 2015]

Ans 51

Question 25

A certain processor supports only the immediate and the direct addressing modes. Which of the following programming language features cannot be implemented on this processor?

- Pointers
- Arrays
- Records
- Recursive procedures with local variable

Explanation

A.Pointers can be implemented only with the use of the indexed addressing mode or indirect addressing mode

B.Same for arrays as option A.

C.Records and arrays are typically types of structure so we will need atleast of the two addressing modes mentioned in option 1.

D.Recursion cant be implemented without stack as we cannot use any register to hold function calls as the no of the function calls would be limited by the number of registers available and we generally do independent compilation.So to use stack and access the data variables we need to use

*(SP+offset) which is again a pointer application.

All four options are correct

Question 26

What does the following program print?

```
#include <stdio.h>
```

```
void f(int *p, int *q)
```

```
{
```

```
    p = q;
```

```
    *p = 2;
```

```
}
```

```
int i = 0, j = 1;
```

```
int main()
```

```

{
f(&i, &j);
printf("%d %d n", i, j);
getchar();
    return 0;
}

```

2010 MCQ || 1-mark |

- A)2 2
- B)2 1
- C)0 1
- D)0 2

Step 1:

f(&i, &j) call:

- p points to i, q points to j.
- $p = q \rightarrow$ p now points to j.
- $*p = 2 \rightarrow$ changes j to 2.

Step 2:

i remains 0, j becomes 2.

Output: 0 2

Question 27

```
#include<stdio.h>
```

```
int f(int *a, int n)
```

```

{
    if(n <= 0) return 0;
    else if(*a % 2 == 0)
        return *a+f(a+1, n-1);
    else
        return *a - f(a+1, n-1);
}

```

```
int main()
```

```

{
    int a[] = {12, 7, 13, 4, 11, 6};

```

```
printf("%d", f(a, 6));
getchar();
return 0;
}
```

2010 MCQ || 2-mark

- A)-9
- B)5
- C)15
- D)19

Explanation

$f()$ is a recursive function which adds $f(a+1, n-1)$ to $*a$ if $*a$ is even. If $*a$ is odd then $f()$ subtracts $f(a+1, n-1)$ from $*a$.

let base address of array is 100 and int takes 2 bytes.

```
f(100, 6)
|
12 + f(102, 5)
  ↓
  7 - f(104, 4)
    ↓
    13 - f(106, 3)
      ↓
      4 + f(108, 2)
        |
        11 - f(110, 1)
          ↓
          6 + F(112, 0)
            ↓
            0
```

So, the final returned value is $12 + (7 - (13 - (4 + (11 - (6 + 0)))))) = 15$

Question28

What does the following fragment of C-program print?

```
char c[] = "GATE2011";
char *p = c;
printf("%s", p + p[3] - p[1]);
```

2011 MCQ || 1-mark

- A)GATE2011
- B)E2011
- C)2011
- D)011

Explanation

p is the starting address of array.

Let consider p address is 100 and p[3] point to A in array and ASCII value of A is 65 and E is 69 so difference of there is 4.

p[3] - p[1] = 4, and p+4 will be pointing to the fifth position in the array 'c'. So printf starts printing from 2 and prints 2011

Question 29

Consider the following recursive C function that takes two arguments.

```
using into foo(unsigned int n, unsigned int r) {
if (n > 0) return ((n%r) + foo(n/r,r));
else return 0;
}
```

What is the return value of the function foo when it is called as foo (513, 2)? **2011 MCQ || 2-mark**

- A)9
- B)8
- C)7
- D)2

Ans D.2

Explanation

```
n  r
foo (513, 2)
|
```

```

1 + foo (256, 2)
  |
0 + foo (128, 2)
  |
0 + foo (64, 2)
  |
0 + foo (32, 2)
  |
0 + foo (16, 2)
  |
0 + foo (8, 2)
    |
    0 + foo (4, 2)
        |
        0 + foo (2, 2)
            |
            0 + foo (1, 2)
                |
                1

```

Question 30

Consider the following recursive C function that takes two arguments

```

int foo(unsigned int n, unsigned int r) {
    if (n > 0)
        return (n%r + foo (n/r, r ));
    else return 0;
}

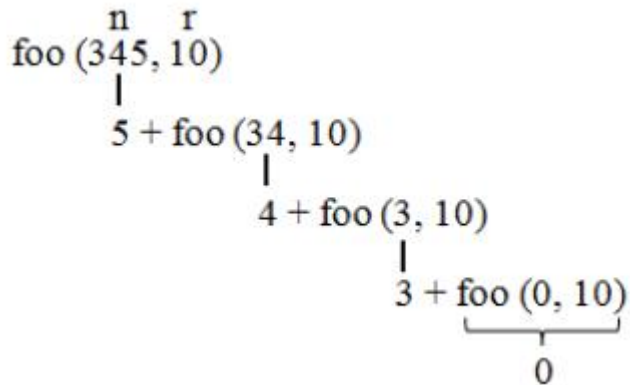
```

What is the return value of the function foo when it is called as foo(345, 10) ? **2011 MCQ || 2-mark**

- A)345
- B)12
- C)5
- D)3

Explanation

B



$$\therefore 5+4+3+0 = 12$$

Question 31

What will be the output of the following C program segment?

```
char inchar = 'A';  
switch (inchar)  
{  
  case 'A':  
    printf ("choice A ");  
  case 'B':  
    printf ("choice B ");  
  case 'C':  
  case 'D':  
  case 'E':  
  default:  
    printf ("No Choice");  
}
```

2012 MCQ || 2-mark

- A)No Choice
- B)Choice A
- C)Choice A Choice B No Choice
- D)Program gives no output as it is erroneous

Explanation

The switch case has no break statements.

- inchar = 'A' → matches case 'A', prints "choice A ".
- Then it **falls through** to case 'B', prints "choice B ".
- Then it **falls through** to case 'C', case 'D', case 'E' (no code, so no print), then default, prints "No Choice".

Output: Choice A Choice B. No choice

Question 32

Consider the following C code segment.

```
int a, b, c = 0;
```

```
void prtFun (void);
```

```
main ()
```

```
{
```

```
    static int a = 1;
```

```
    prtFun ();
```

```
    a + = 1;
```

```
    prtFun ();
```

```
    printf("\n %d %d ", a, b);
```

```
}
```

```
Void prtFun (void)
```

```
{
```

```
    static int a = 2;
```

```
    int b = 1;
```

```
    a + = + + b;
```

```
    printf (" \n %d %d ", a, b);
```

```
}
```

What output will be generated by the given code segment? **2012 MCQ || 2-mark**

- A) 3 1

4 1

4 2

- B)

4 2

6 1

6 1

- C)

4 2

6 2

2 0

- D)

3 1

5 2

5 2

Explanation

Global variables:

int a, b, c = 0; → a = 0, b = 0, c = 0 (global scope).

Inside main:

static int a = 1; (local to main, hides global a).

First call prtFun():

Inside prtFun:

- static int a = 2; (local to prtFun, separate from main's a and global a).
- int b = 1; (local to prtFun).

a += ++b; → ++b makes b = 2, so a = 2 + 2 = 4.

Print: 4 2 (first output line).

Back to main: a += 1 → main's local a was 1, now 2.

Second call prtFun():

Inside prtFun:

- static int a retains its value from last call: a = 4.
- int b = 1 again (reinitialized local).

a += ++b → ++b → b = 2, so a = 4 + 2 = 6.

Print: 6 2 (second output line).

Back to main:

printf("\n %d %d", a, b) → a is main's local a = 2, b is global b (never changed) = 0.

Print: 2 0 (third output line).

final Out put : 4 2

6 2

2 0

Question 33

Consider the following function:

```
int unknown(int n) {  
    int i, j, k = 0;  
    for (i = n/2; i <= n; i++)  
        for (j = 2; j <= n; j = j * 2)  
            k = k + n/2;  
    return k;  
}
```

The **return** value of the function is__

2013 MCQ || 2-mark |

- A

$\Theta(n^2)$

- B

$\Theta(n^2 \log n)$

- C

$\Theta(n^3)$

- D

$\Theta(n^3 \log n)$

Explanation

Outer loop runs for $(n/2)$ times and inner loop runs for $(\log n)$ times.

So, the total number of times loop runs is $(n/2 \log n)$.

So, the final k value will be $n/2 * (n/2 \log n) = O(n^2 \log n)$

$= (n/2 + 1) \cdot n/2 \cdot \log n$

$= (n^2 \log n)$

Question 34

What is the return value of $f(p,p)$, if the value of p is initialized to 5 before the call? Note that the first parameter is passed by reference, whereas the second parameter is passed by value.

```
int f(int &x, int c) {
```

```

c = c - 1;
if (c==0) return 1;
x = x + 1;
return f(x,c) * x;
}

```

2013 || MCQ || 1-MARK

- A

3024

- B

6561

- C

55440

- D

161051

Explanation

Step-by-step without long paragraphs:

1. **First call:** $x = p = 5$, $c = 5$
 $c = 4$, p becomes 6 (since x is reference to p).
 Return $f(p, 4) * p$.
2. **Second call:** $p = 6$, $c = 4$
 $c = 3$, p becomes 7.
 Return $f(p, 3) * p$.
3. **Third call:** $p = 7$, $c = 3$
 $c = 2$, p becomes 8.
 Return $f(p, 2) * p$.
4. **Fourth call:** $p = 8$, $c = 2$
 $c = 1$, p becomes 9.
 Return $f(p, 1) * p$.
5. **Fifth call:** $p = 9$, $c = 1$
 $c = 0$, return 1.

Unwinding recursion:

- Step 4: $f(p, 1) * p = 1 * 9 = 9$
- Step 3: $f(p, 2) * p = 9 * 9 = 81$
- Step 2: $f(p, 3) * p = 81 * 9 = 729$
- Step 1: $f(p, 4) * p = 729 * 9 = 6561$

Final answer: 6561

Question 35

Let A be a square matrix of size $n \times n$. Consider the following pseudocode. What is the expected output? **2014 SET 3 || MCQ || 2-mark**

```
C = 100;

for i = 1 to n do
  for j = 1 to n do
  {
    Temp = A[i][j] + C;
    A[i][j] = A[j][i];
    A[j][i] = Temp - C;
  }
  for i = 1 to n do
    for j = 1 to n do
      output (A[i][j]);
```

- A

The matrix A itself

- B

Transpose of the matrix A

- C

Adding 100 to the upper diagonal elements and subtracting 100 from lower diagonal elements of A

- D

None of the above

Explanation

1. **Manipulation Block:** The three lines inside the innermost loop:
 2. $\text{Temp} = A[i][j] + C;$
 3. $A[i][j] = A[j][i];$
 4. $A[j][i] = \text{Temp} - C;$

This sequence performs a **swap** of $A[i][j]$ and $A[j][i]$. The constant $C=100$ is added and then subtracted, having no net effect on the values themselves.

2. **Loop Structure:** The nested loops iterate over *all* pairs of indices (i,j) from 1 to n .

3. Net Effect:

- For any pair where $i \neq j$ (off-diagonal elements), the loops process the pair **twice**: once as (i,j) and once as (j,i) .
- A swap operation performed twice on the same pair of memory locations returns the elements to their **original positions and values**.
- For $i=j$ (diagonal elements), the element is swapped with itself, resulting in no change.

4. **Final Output:** Since the matrix A is restored to its initial state after the first set of loops, the second set of loops, which prints $A[i][j]$ in row-major order, prints the **original matrix A**.

Therefore, the expected output is **The matrix A itself**.

The correct option is: **The matrix A itself**

Question 36

Consider the following function

```
#include <math.h>
```

```
double f(double x){  
    if (fabs(x*x - 3) < 0.01) return x;  
    else return f(x/2 + 1.5/x);  
}
```

Give a value q (to 2 decimals) such that $f(q)$ will return q : _____. **2014 SET 2 || NAT || 2-mark**

- A

1.73

Explanation

The stopping condition is:

We need a value q (to 2 decimal places) that satisfies:

This means q^2 must be between 2.99 and 3.01:

Taking the square root gives the range:

The value $3 \approx 1.73205$ is in this range. Rounding this to two decimal places gives **1.73**.

Verification: $1.732^2 = 2.9929$. The absolute difference is $|2.9929 - 3| = 0.0071$, which is less than 0.01.

The value q is **1.73**.

Question 37

Consider the function shown below :

```
int func (int num) {  
    int count = 0;  
    while (num) {  
        count ++;  
        num >>= 1;  
    }  
    return (count);  
}
```

The value returned by func (435) is _____. **2014 SET 2 || NAT || 1-mark**

Correct Answer! 9

Explanation

Step 1: Binary of 435

435 in binary is 110110011 (9 bits).

Step 2: Loop working

while(num) runs until num becomes 0.

Each iteration:

- count increments by 1
- num >>= 1 shifts right by 1 bit

Step 3: Counting shifts

Start: num = 435 (binary 110110011)

After 1 shift: 217

After 2 shifts: 108

After 3 shifts: 54

After 4 shifts: 27

After 5 shifts: 13

After 6 shifts: 6

After 7 shifts: 3

After 8 shifts: 1

After 9 shifts: 0 → stop

Total shifts = 9 → count = 9

Final answer: 9

Question 338

Which of the following statements are CORRECT?

- 1.Static allocation of all data areas by a compiler makes it impossible to implement recursion.
- 2Automatic garbage collection is essential to implement recursion.
- 3.Dynamic allocation of activation records is essential to implement recursion.
- 4.Both heap and stack are essential to implement recursion. **2014 SET 3 || MCQ || 1-mark**

- A

1 and 2 only

- B

2 and 3 only

- C

3 and 4 only

- D

1 and 3 only

Explanation

1) Static allocation of all data areas by a compiler makes it impossible to implement recursion. True, dynamic allocate of memory is required for function call stack as number of calls is not known advance for recursive functions. 2) Automatic garbage collection is essential to implement recursion. False, Automatic garbage collection is not essential. 3) Dynamic allocation of activation records is essential to implement recursion. True, as number of calls or number of activation records is not known advance for recursive functions. 4) Both heap and stack are essential to implement recursion. Heap is not needed for function calls. It is generally used for dynamic memory allocation by user (or programmer)

Question 39

Consider the C function given below.

```
int f(int j)
{
    static int i = 50;
    int k;
    if (i == j)
    {
        printf("something");
        k = f(i);
        return 0;
    }
}
```

```

}
else return 0;
}

```

Which one of the following is TRUE? **2014 SET 2 || MCQ || 1-mark**

- A

The function returns 0 for all values of j.

- B

The function prints the string something for all values of j

- C

The function returns 0 when j = 50.

- D

The function will exhaust the runtime stack or run into an infinite loop when j = 50.

Incorrect Answer!

Explanation

f(50)

j = 50

static int i = 50 (declare and define one time in whole program)

if condition is true

print something

k = f(50)

it will keep running and will cause stack overflow

Question 40

Consider the following program in C language:

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    int i;
```

```
    int *pi = &i;
```

```
    scanf("%d", pi);
```

```
printf("%d\n", i+5);  
}
```

Which one of the following statements is TRUE? **2014 SET 1 || MCQ || 1-mark**

- A

Compilation fails.

- B

Execution results in a run-time error.

- C

On execution, the value printed is 5 more than the address of variable i.

- D

On execution, the value printed is 5 more than the integer value entered.

Explanation

D

int i; // Initially i takes the Garbage value

int *pi = &i; // pi is a pointer which stores the address of i.

scanf (pi); // pi = &i (we rewrite the garbage value with our values) say x = 5

printf (i+5); // i+5 = x+5 = 5+5 = 10

Hence on execution, the value printed is 5 more than the integer value entered.

Question 41

Consider the following C program segment.

```
# include <stdio.h>
```

```
int main( )
```

```
{
```

```
char s1[7] = "1234", *p;
```

```
p = s1 + 2;
```

```
*p = '0' ;
```

```
printf ("%s", s1);
```

```
}
```

What will be printed by the program? **2015 SET 3 || MCQ || 1-mark**

- A

12

- B

120400

- C

1204

- D

1034

Explanation

`char s1[7] = "1234" → s1 contains: '1' '2' '3' '4' '\0' ...`

`p = s1 + 2 → p points to s1[2] which is '3'`

`*p = '0' → s1[2] becomes '0'`

Now `s1 = '1' '2' '0' '4' '\0'`

`printf("%s", s1)` prints 1204

Answer: 1204

Question 42

Consider the following function written the C programming language. **2015 SET 2 || MCQ || 1-mark**

```
void foo (char *a) {
    if (*a && *a != ' ') {
        foo (a+1);
        putchar (*a);
    }
}
```

The output of the above function on input "ABCD EFGH" is

- A

ABCD EFGD

- B

ABCD

- C

HGFE DCBA

- D

DCBA

Explanation

Step-by-step:

1. `foo("ABCD EFGH")` → `*a = 'A'` → call `foo("BCD EFGH")`
2. `foo("BCD EFGH")` → `*a = 'B'` → call `foo("CD EFGH")`
3. `foo("CD EFGH")` → `*a = 'C'` → call `foo("D EFGH")`
4. `foo("D EFGH")` → `*a = 'D'` → call `foo(" EFGH")`
5. `foo(" EFGH")` → `*a = ' '` → condition fails → return

Now unwinding:

- Step 4: `putchar('D')` → prints D
- Step 3: `putchar('C')` → prints C
- Step 2: `putchar('B')` → prints B
- Step 1: `putchar('A')` → prints A

Output: DCBA

Answer: DCBA

Question 43

What is the output of the following C code? Assume that the address of x is 2000 (in decimal) and an integer requires four bytes of memory.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    unsigned int x[4][3] = {{1, 2, 3}, {4, 5, 6},
```

```
    {7, 8, 9}, {10, 11, 12}};
```

```
    printf("%u, %u, %u", x+3, *(x+3), *(x+2)+3);
```

```
}
```

2015 SET 1 || MCQ || 1-mark

- A

2036, 2036, 2036

- B

2012, 4, 2204

- C

2036, 10, 10

- D

2012, 4, 6

Explanation

Given:

$x[4][3] \rightarrow$ 4 rows, 3 columns.

Base address of $x = 2000$.

Size of unsigned int = 4 bytes.

1. $x + 3$

x is of type unsigned int (*)[3] (pointer to array of 3 ints).

$x + 3 =$ address of $x[3]$ (4th row).

Row size = $3 \times 4 = 12$ bytes.

So $x + 3 = 2000 + 3 \times 12 = 2000 + 36 = \mathbf{2036}$.

2. $*(x + 3)$

$*(x + 3) = x[3] =$ address of first element of 4th row ($x[3][0]$).

Address of $x[3][0] = 2036$ (same as $x + 3$ numerically but type is unsigned int *).

So value printed = **2036**.

3. $*(x + 2) + 3$

$*(x + 2) = x[2] =$ address of $x[2][0] = 2000 + 2 \times 12 = 2024$.

$*(x + 2) + 3 = \&x[2][3]$? Wait, $x[2]$ has only 3 columns (0,1,2), so $x[2] + 3 =$ address of $x[2][3]$ is out of bounds for row 2, but in C it's just pointer arithmetic:

$x[2] + 3 = 2024 + 3 \times 4 = 2024 + 12 = \mathbf{2036}$.

So all three expressions give:

2036, 2036, 2036

Output: 2036, 2036, 2036

Question 44

Consider the following recursive C function. If get(6) function is being called in main() then how many times will the get() function be invoked before returning to the main()? **2015 SET 3 || MCQ || 2-mark**

```
void get (int n)
```

```
{
```

```
    if (n < 1) return;
```

```

get(n-1);
get(n-3);
printf("%d", n);
}

```

- A

15

- B

25

- C

35

- D

45

Explanation

Let $N(n)$ be the number of invocations for $get(n)$. The recurrence relation is:

with base cases:

- $N(-2)=1$
- $N(-1)=1$
- $N(0)=1$
- $N(1)=1+N(0)+N(-2)=1+1+1=3$
- $N(2)=1+N(1)+N(-1)=1+3+1=5$
- $N(3)=1+N(2)+N(0)=1+5+1=7$
- $N(4)=1+N(3)+N(1)=1+7+3=11$
- $N(5)=1+N(4)+N(2)=1+11+5=17$
- $N(6)=1+N(5)+N(3)=1+17+7=25$

The $get()$ function will be invoked **25** times.

Question 45

Consider the following C program. **2015 SET 3 || NAT || 2-mark**

```
# include<stdio.h>
```

```
int main( )
```

```
{
```

```
    static int a[] = {10, 20, 30, 40, 50};
```

```

static int *p[] = {a, a+3, a+4, a+1, a+2};

int **ptr = p;

ptr++;

printf("%d%d", ptr - p, **ptr);

}

```

The output of the program is _____

- A

140

Explanation

Initial arrays:

- $a[] = \{10, 20, 30, 40, 50\}$
- $p[] = \{a, a+3, a+4, a+1, a+2\}$
That means:
 $p[0] = \&a[0] \rightarrow 10$
 $p[1] = \&a[3] \rightarrow 40$
 $p[2] = \&a[4] \rightarrow 50$
 $p[3] = \&a[1] \rightarrow 20$
 $p[4] = \&a[2] \rightarrow 30$
- $ptr = p \rightarrow ptr$ points to $p[0]$

Step: $ptr++$

ptr now points to $p[1]$

$ptr - p$

= (address of $p[1]$ - address of $p[0]$) in units of int^*
= 1

$**ptr$

$*ptr = p[1] = a+3 \rightarrow$ points to $a[3] = 40$

So $**ptr = 40$

Output: 1 and 40 $\rightarrow$ 140

Question 46

Consider the following C program: **2015 SET 3 || NAT || 2-mark**

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
int i, j, k = 0;
```

```

j = 2 * 3 / 4 + 2.0 / 5 + 8 / 5;
k -= --j;
for (i = 0; i < 5; i++)
{
    switch(i + k)
    {
        case 1:
        case 2: printf("\n%d", i + k);
        case 3: printf("\n%d", i + k);
        default: printf("\n%d", i + k);
    }
}
return 0;
}

```

TOTAL NUMBER OF TIMES PRINTF EXECUTED ?

- A

10

Question 47

Consider the following C program. The output of the program is _____. **2015 SET 3 || NAT || 2-mark**

```

#include <stdio.h>

int f1(void);
int f2(void);
int f3(void);

int x = 10;

int main()
{
    int x = 1;
    x += f1() + f2() + f3() + f2();
    printf("%d", x);

    return 0;
}

```

```

int f1()
{
    int x = 25;

    x++;

    return x;
}

int f2( )
{
    static int x = 50;

    x++;

    return x;
}

int f3( )
{
    x *= 10;

    return x;
}

```

- A

230

Correct Answer!

Explanation

Global x = 10

main() local x = 1

f1()

local x = 25 → x++ → return 26

First f2()

static x = 50 → x++ → return 51 (static now 51)

f3()

global x = 10 → x *= 10 → global x = 100 → return 100

Second f2()

static x = 51 → x++ → return 52 (static now 52)

Expression in main:

x += f1() + f2() + f3() + f2()

= x += 26 + 51 + 100 + 52

= x += 229

Local x was 1 → now 1 + 229 = 230

Output: 230

Question 48

Consider the following C function:

```
int fun (int n) {  
    int x = 1, k;  
    if (n == 1) return x;  
    for (k = 1; k < n; ++k)  
        x = x + fun (k) * fun (n - k);  
    return x;  
}
```

The return value of fun (5) is _____. **2015 SET 2 || NAT || 1-mark**

- A

51

Question 49

The value printed by the following program is _____. **2016 SET 2 || NAT || 2-mark**

```
void f(int* p, int m) {  
    m = m + 5;  
    *p = *p + m;  
    return;  
}  
  
void main() {  
    int i=5, j=10;  
    f(&i, j);  
    printf("%d", i+j);  
}
```

- A

30

Explanation

Initial:

i = 5, j = 10

Call f(&i, j):

- p points to i
- m = 10 (local copy)
- m = m + 5 → m = 15
- *p = *p + m → i = 5 + 15 = 20
- Return

After call:

i = 20, j = 10 (unchanged because j was passed by value)

i + j = 20 + 10 = 30

Output: 30

Question 50

The following function computes X^Y for positive integers X and Y.

```
int exp (int X, int Y) {  
    int res = 1, a = X, b = Y;  
    while ( b != 0 ){  
        if ( b%2 == 0)  
        {  
            a = a*a;  
            b = b/2;  
        }  
        else  
        {  
            res = res*a;  
            b = b-1;  
        }  
    }  
    return res;  
}
```

Which one of the following conditions is **TRUE** before every iteration of the loop? **2016 SET 2 || MCQ || 2-mark**

- A

$$X^Y = a^b$$

- B

$$(\text{res} * a)^Y = (\text{res} * X)^b$$

- C

$$X^Y = \text{res} * a^b$$

- D

$$X^Y = (\text{res} * a)^b$$

Explanation

```
int exp (int X, int Y)
{
    int res = 1, a = X, b = Y;
    while (b != 0)
    {
        if (b%2 == 0)
        {
            a = a*a;
            b = b/2;
        }
        else
        {
            res = res*a;
            b = b - 1;
        }
    }
    return res;
}
```

From that explanation part you can understand the exponent operation, but to check the conditions, first while iteration is enough.

$x = 2, y = 3, \text{res} = 2, a = 2, b = 2.$

Only (C) satisfies these values.

$$x^y = \text{res} * a^b$$

$$2^3 = 2 * 2^2 = 8$$